

1. Introduction

This document outlines the technical architecture, deployment model, and operational procedures for Nyx, a conversational agent built on ElizaOS and hosted on Google Cloud Platform (GCP). It is intended for engineering, operations, and security teams to provide a comprehensive understanding of Nyx's construction, deployment, and ongoing maintenance in a production environment.

1.1. Background

ElizaOS framework (TypeScript/Bun runtime) to manage multi-channel conversational experiences. This is achieved through various pluggable integrations, including OpenAI, Telegram, Discord, and X/Twitter. The service's state is maintained in Cloud SQL for PostgreSQL, while stateless compute operations are executed on Cloud Run. Images are stored in Artifact Registry, and builds are coordinated using Cloud Build.

1.2. Objectives

- Provide a clear and reproducible blueprint for deploying and operating Nyx in GCP. Document system components, configuration, and environment variables.
- Describe networking, security, and secrets management approaches.
- Establish baselines for monitoring, logging, and alerting.
- Outline near-term hardening and CI/CD improvements.

1.3. Scope

- Production deployment on GCP (Cloud Run, Artifact Registry, Cloud SQL, Cloud Build).
- Runtime configuration (ports and environment variables).
- Observability (Cloud Logging/Monitoring).
- Security baselines (DB access controls, Secret Manager plan).

1.4. Stakeholders & Audience

- **Engineering** (backend, DevOps) — build, deploy, and maintain the service.
- **Security** — validate access controls, secrets handling, and data protection.
- **Product/Operations** — understand SLAs, dependencies, and the change process.

2. Architecture Overview

The ElizaOS Agent architecture on Google Cloud Platform (GCP) follows a modular, serverless, and secure design that enables scalability, maintainability, and clear separation of responsibilities. Figure 1 illustrates the major components and how data flows between them.

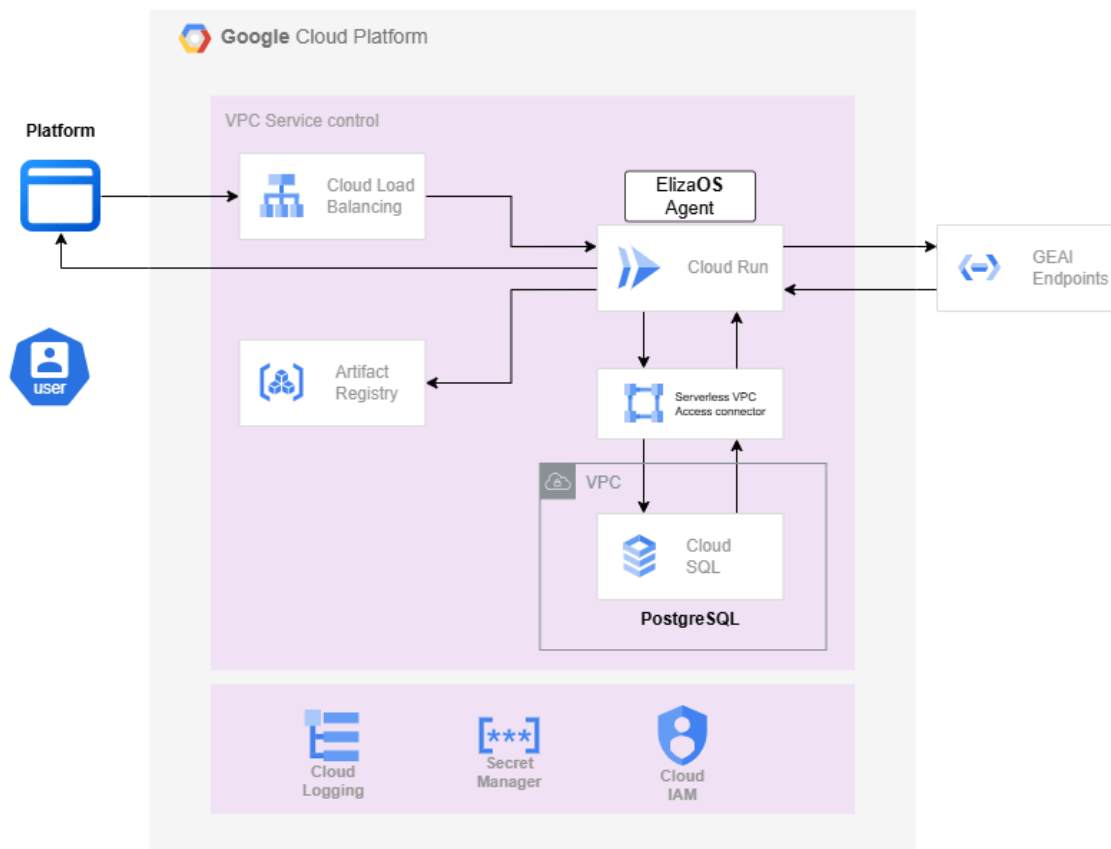


Figure 1: ElizaOS Agent Architecture on GCP

2.1. Request Flow Description

1. User Interaction and Entry Point

End users or external channels (e.g., Discord, Telegram, or X) send HTTPS requests to the *ElizaOS Agent* endpoint.

Traffic is first received by *Cloud Load Balancing*, which serves as the unified entry point for all incoming requests. It distributes traffic efficiently across available *Cloud Run* instances and ensures regional redundancy and performance.

2. Application Layer – ElizaOS Agent on Cloud Run

The *ElizaOS Agent* runs as a containerized service on *Cloud Run*, leveraging a Bun/TypeScript runtime and the *ElizaOS framework*.

Cloud Run instances are stateless and scale automatically based on the number of incoming requests. Each instance processes user interactions, executes business logic, and manages plugin calls (e.g., OpenAI, Anthropic, or GEAI integrations).

When the service starts or scales up, it retrieves its container image from *Artifact Registry*, ensuring version-controlled and immutable deployments.

3. Secure Network Connectivity – Serverless VPC Access

Cloud Run establishes a secure connection to internal resources via a *Serverless VPC Access connector*, linking to the Virtual Private Cloud (VPC). This connector facilitates

private communication with internal services (like PostgreSQL) and shields them from public internet exposure.

4. **Data Persistence – Cloud SQL for PostgreSQL**

All persistent data (conversation states, user sessions, configuration, logs) is stored in *Cloud SQL for PostgreSQL*.

The database is deployed with private IP access, meaning it can only be reached through the VPC connector. This ensures data security and compliance while maintaining low-latency connections between Cloud Run and the database.

5. **External Integrations – GEAI Endpoints**

The ElizaOS Agent communicates with *GEAI (Globant Enterprise AI)* and other AI service providers through secure outbound HTTPS requests. These APIs extend the agent's reasoning, context enrichment (knowledge), and conversational capabilities.

6. **Security and Configuration Management**

Sensitive information such as database URLs, API keys, and provider credentials are stored in *Secret Manager*.

Access to these secrets is controlled through *Cloud IAM*, enforcing the principle of least privilege.

7. **Monitoring and Operations**

Cloud Logging automatically captures application logs, while *Cloud Monitoring* provides real-time insights into CPU usage, memory, request latency, and error rates.

Together, these services enable proactive alerting, issue diagnosis, and performance tuning for the ElizaOS Agent.

3. ElizaOS

ElizaOS is an advanced, modular open-source platform designed for building and deploying intelligent agents. Its architecture allows for flexibility and customization through npm plugins, enabling users to easily integrate various capabilities such as model providers, vector stores, and social networks without being tied to a specific vendor.

ElizaOS works with an architecture based on DI (dependency injection) called plugins, where a plugin is a bundle of capabilities you drop into an agent. There are two plugins that are required: SQL for storing, and Bootstrap as message handler and event system for elizaOS agent. It provides essential functionality for message processing and basic agent operations.

Plugin

- ☐ Actions
- ☐ Providers
- ☐ Evaluators
- ☐ Services
- ☐ Adapters
- ☐ Models
- ☐ Routes
- ☐ Events

For each of those pieces, ElizaOS/core decides when to execute.

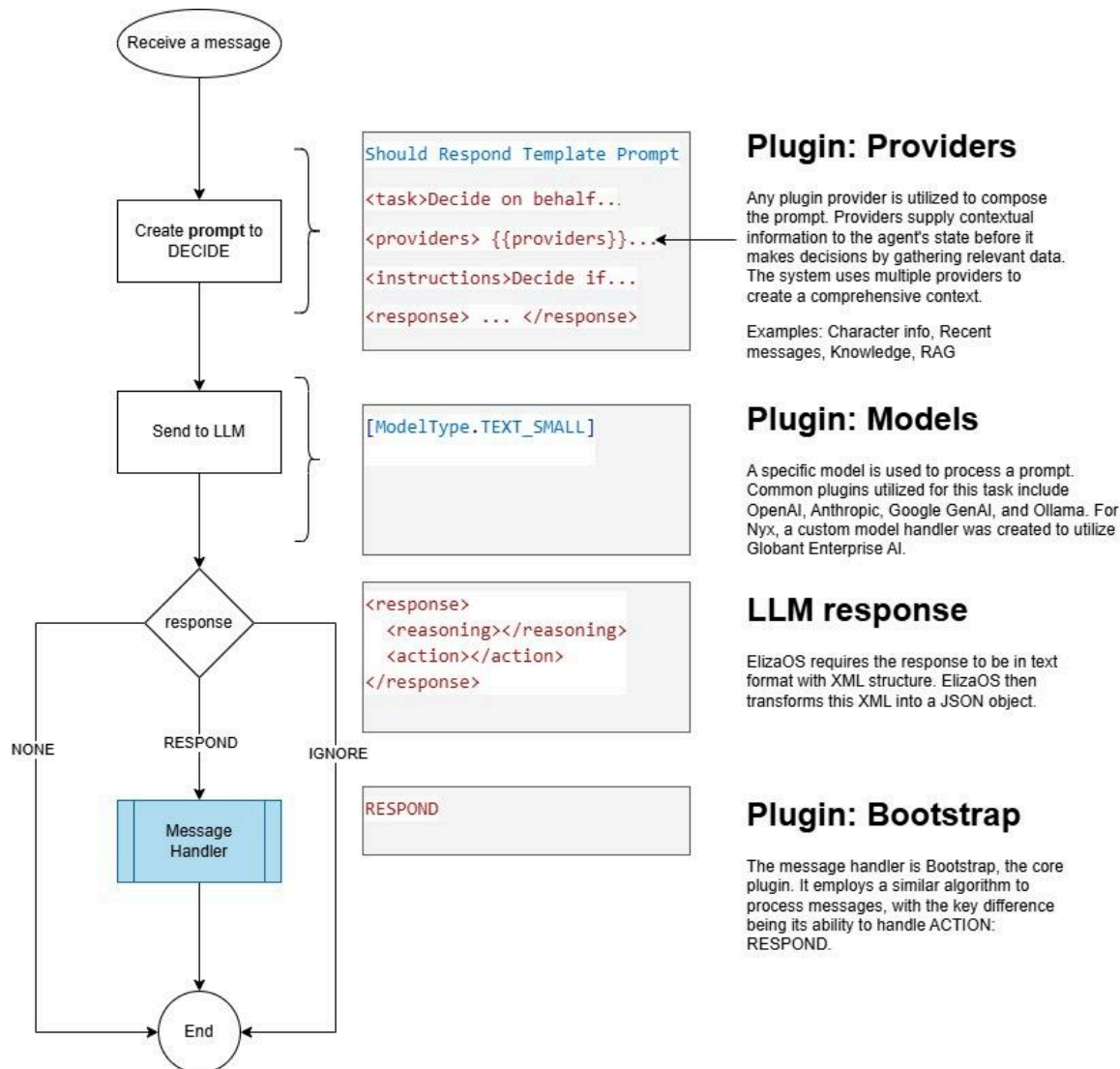


Figure 2: Discord, Telegram message on ElizaOS

The first step when ElizaOS receives a message from Discord or Telegram is to determine whether it should ignore the message or respond to it. ElizaOS will ignore the message if the conversation does not fall within its knowledge base or if someone is asking about another person.

This decision-making process is facilitated by a large language model (LLM). Following this, ElizaOS prepares a prompt template that incorporates Nyx's personality, previous messages, actions, and other relevant information to provide a comprehensive context for the LLM to make an informed decision.

The prompt to DECIDE is generated from a prompt template that includes mustache-type variables and providers. The configuration and template details will be discussed in the Prompt section.

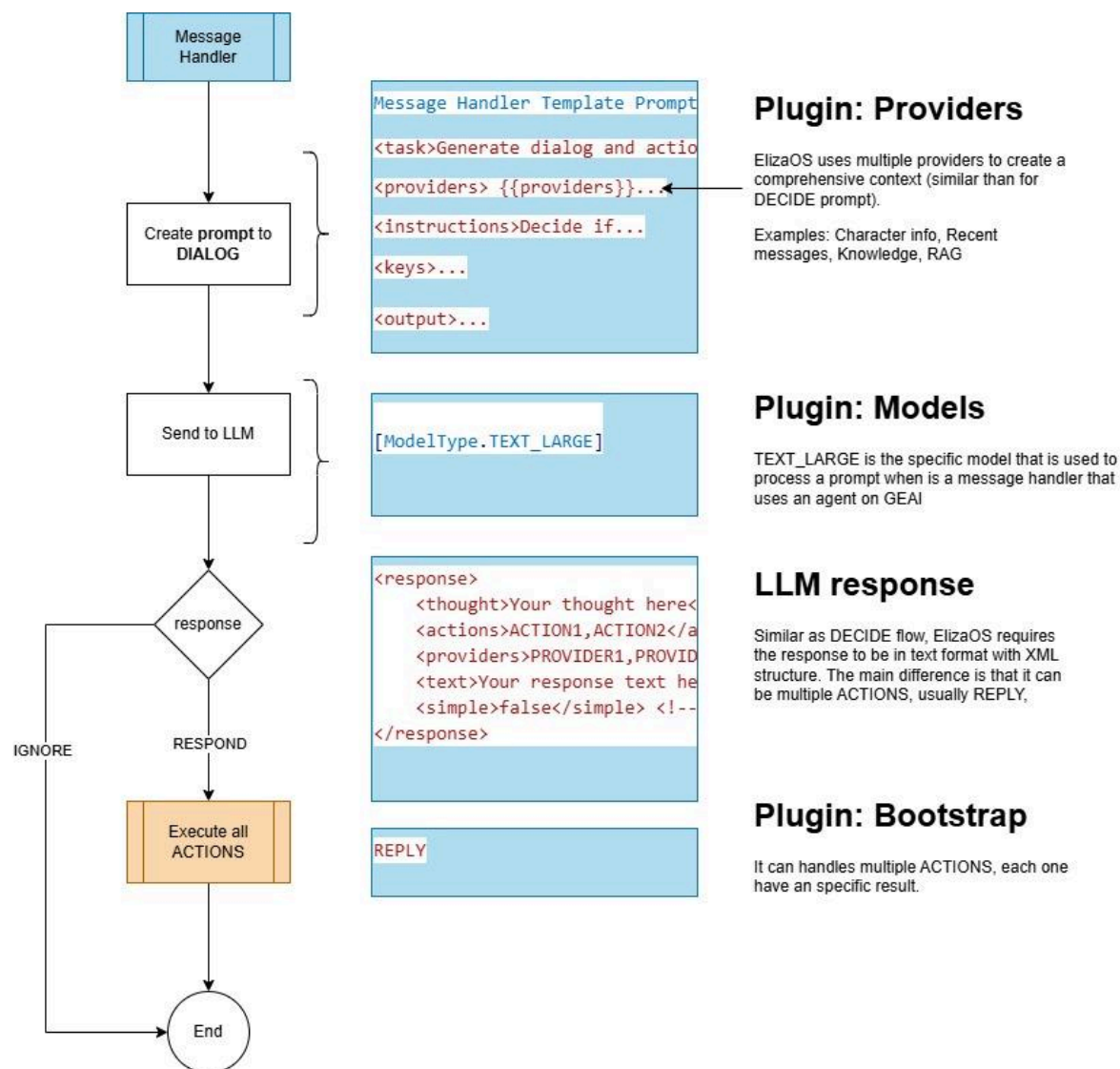


Figure 3: Common message handler by ElizaOS.

In the second step, when ElizaOS needs to respond. In Figure 3, when ElizaOS encounters a general question, it prepares the **prompt** with all relevant context using "*Generate dialog and actions.*" meaning using **messageHandlerTemplate**. With this prompt the LLM decides the next Actions like REPLY (direct message to user), use WEB_SEARCH_OR_RAG (to go through Search Gate service) and so on.

The simplest action is a straightforward REPLY, but there can be multiple actions as well. In this scenario, the LLM being utilized is the GEAI-agent.

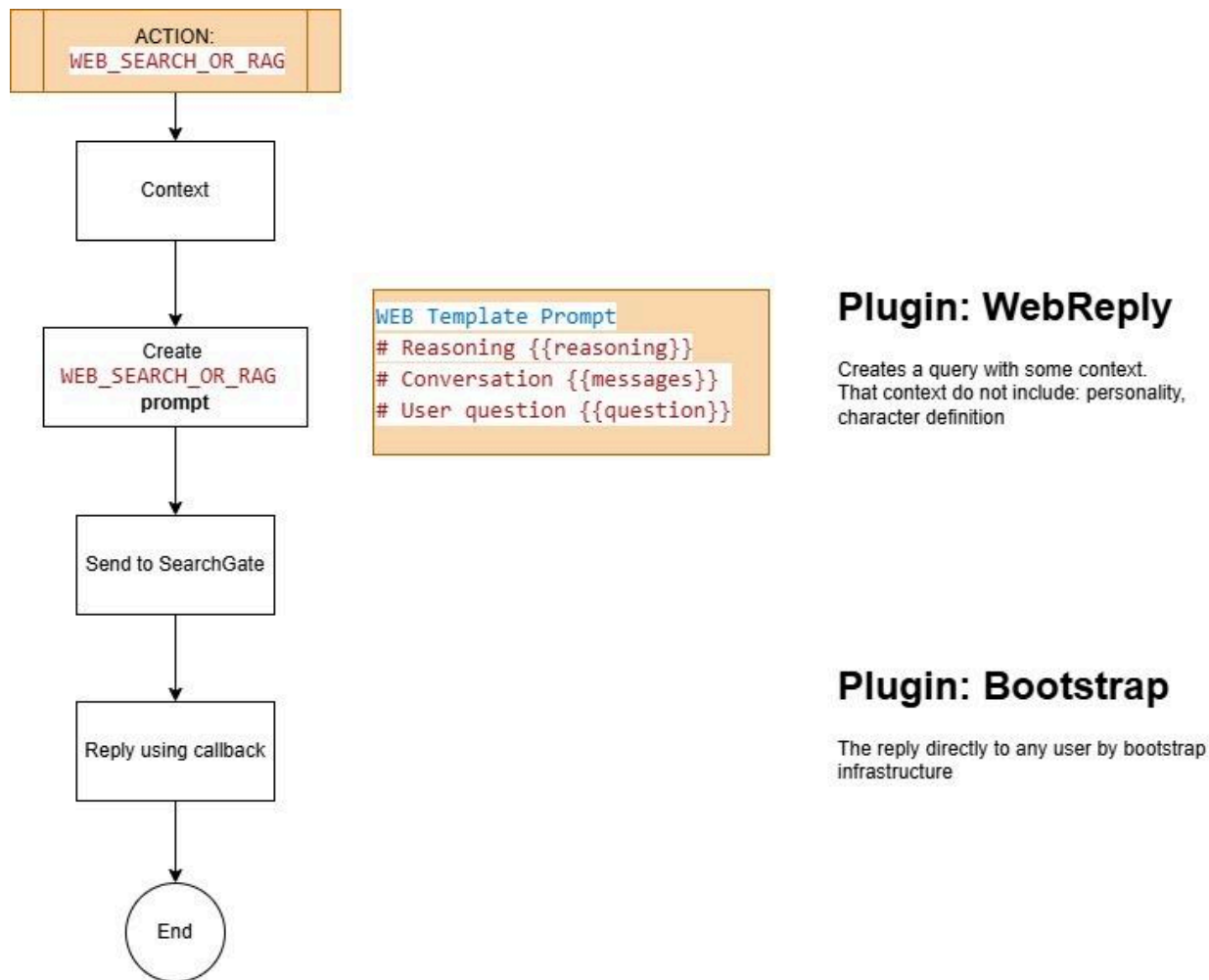


Figure 4: Discord, Telegram message on ElizaOS

The third way to answer the user's question is through a custom flow in GEAI. The GEAI assistant determines when to use the RAG and when to utilize web results to provide an answer. In this case, ElizaOS only sends some contextual information.

This custom action responds "directly" to the user request, meaning that no further actions are required.

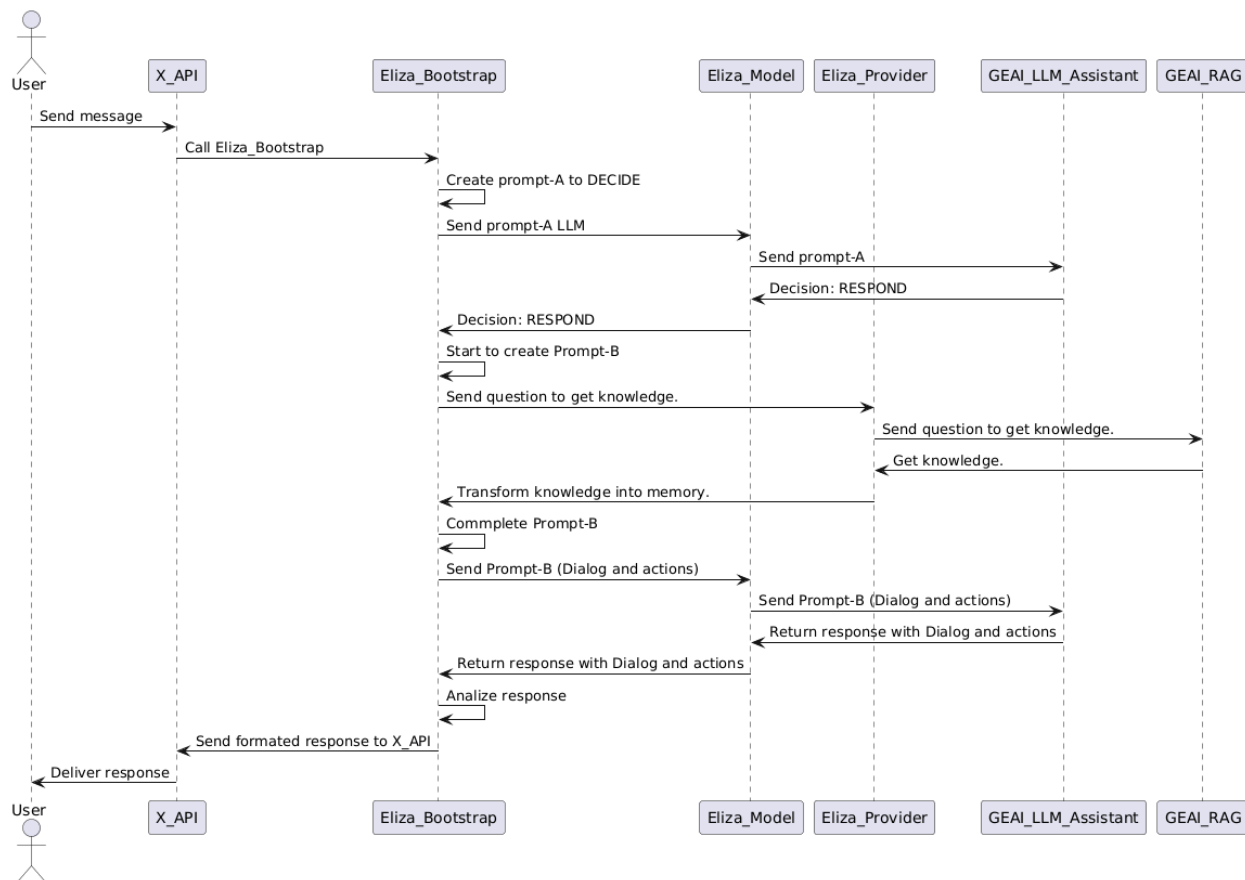


Figure 5: Message with assistant that includes Knowledge (RAG)

When the response is through an assistant and knowledge, the way is a little bit different, because the knowledge must be inserted into the prompt B. So, it requires requesting the RAG information in order to add that information into the Knowledge section.

4. Plugins

To customize the behavior of ElizaOS, several plugins with special functionalities have been created. The most significant of these is the connection to Globant Enterprise AI (GEAI), but there are also plugins for connecting with RAG and Web Search. Additionally, there are plugins that handle actions such as custom error handling, custom ignore functionality, and special trimming of message lengths.

4.1. Custom Error Plugin

The Custom Error Plugin manages application errors within the ElizaOS Agent to ensure graceful degradation and provide custom feedback. In the standard flow of Eliza, users do not receive any notification after an error occurs. To address this, the plugin adds a custom action that sends a fixed message to the user, informing them of the error.

Trigger and Flow:

The plugin is triggered immediately upon an error event during communication, typically when an upstream dependency fails (e.g., HTTP timeout, guardrails, etc.). This mechanism ensures consistent and user-friendly error handling while maintaining operational visibility.

4.2. Custom messages length plugin

This plugin modifies the behavior of the Messages provider. By default, it retrieves all messages or up to 31 messages, even though there is an environment variable intended to limit the maximum number of messages. However, this variable does not function correctly in all versions. To address this issue, the plugin enforces the use of the **CONVERSATION_LENGTH** variable to set the maximum number of messages included in the prompts.

4.3. Custom Ignore plugin

The Custom Ignore Plugin introduces a special action. By default, when the flow decides to ignore a conversation, it does nothing. However, there are times when it is important to inform the user that we couldn't answer a question, particularly when the question infringes on certain restrictions related to policies or character scope.

For this reason, the plugin ensures that the user is notified of the restriction instead of simply not receiving any message.

4.4. RAG plugin

The RAG Plugin introduces two providers.

The first provider retrieves the message, capturing not only the text but also all other properties, and stores them in a custom tag. This special feature allows for the use of that information in subsequent steps.

The second provider calls the GEAI-RAG via its API, transforming its results into a memory format and text for use. This functions similarly to a Knowledge provider, but instead of the knowledge being stored locally, it is part of the RAG, which includes web scraping of documentation, wikis, and documents related to Midnight.

Custom reply plugin

This plugin adds the capability to respond to a message from the user with the special assistant on GEAI, this assistant can search on the web, search on RAG or a simple response.

Globant Enterprise AI plugin

GEAI plugin works instead of OpenAI plugin. This is the core communication with the LLM

4.5. Globant Enterprise AI Plugin

The Globant Enterprise AI (GEAI) Plugin is a critical component that effectively replaces the standard OpenAI plugin within the ElizaOS framework. It serves as the core communication layer between the ElizaOS Agent and the specialized GEAI infrastructure.

Core Functionality:

- **LLM Communication Hub:** The primary role of this plugin is to handle all outbound requests to the GEAI endpoints. It manages the secure transmission of prompts (including Prompt A and Prompt B structures) and the receipt of responses from the LLM, effectively serving as the heart of the agent's intelligence.
- **Prompt Translation and Dispatch:** It is responsible for taking the XML-structured prompts generated by ElizaOS (which incorporate conversation history, personality, actions, and instructions) and correctly formatting/dispatching them to the appropriate GEAI service, such as the GEAI-agent for action/dialogue generation or the custom assistants.
- **Integration with Custom GEAI Flows:** The GEAI plugin enables the use of specialized GEAI assistants that incorporate advanced features like web search capabilities or integrated RAG (Retrieval-Augmented Generation) based on Globant's internal knowledge base (as described in Figure 5). It ensures that contextual information is passed correctly for these custom flows.

4.6. Role in the Architecture:

By abstracting the LLM interaction through the GEAI plugin, the Nyx architecture achieves several goals:

1. **Vendor Agnosticism (Framework Level):** Although specifically tied to GEAI, it maintains the modularity of ElizaOS by slotting into the provider structure where a generic LLM service would reside.
2. **Security and Compliance:** It manages the secure authentication and authorization required to access the Globant internal AI services, adhering to enterprise security standards.

3. Performance: It leverages the specialized GEAI infrastructure optimized for enterprise use cases.

5. Prompts

Prompts are an important part of the system since it defines the task along with all information. The prompt is not static, ElizaOs creates a prompt from a template.

5.1. How use and configure a prompt template in ElizaOs



Figure 5: Prompt creation from Template

Types of prompt templates

Main Template Types, each template serves a specific purpose in the message flow:

- **shouldRespondTemplate** → Decision point (RESPOND/IGNORE/STOP) controls when the agent decides to respond a conversation
- **messageHandlerTemplate** → Response generation
- **reflectionTemplate** → Learning and memory updates, used for post-interaction analysis and learning
- **postCreationTemplate** → Social media content generation

How configure in Character

These templates are configured in the character's templates property.

```

JavaScript
Character {

  name: string,

  ...

  templates: {
  
```

```
messageHandlerTemplate: string,

shouldRespondTemplate: string,

reflectionTemplate: string,

postCreationTemplate: string

}
```

When not explicitly defined, Bootstrap uses a default prompt template, with a different default for each template type.

Here is an example of the default **messageHandlerTemplate** provided by the Bootstrap plugin:

XML

```
<task>Generate dialog and actions for the character
{{agentName}}.</task>
```

```
<providers>
```

```
{{providers}}
```

```
</providers>
```

```
<instructions>
```

```
Write a thought and plan for {{agentName}} and decide what actions
to take. Also include the providers that {{agentName}} will use to
have the right context for responding and acting, if any.
```

```
IMPORTANT ACTION ORDERING RULES:
```

- Actions are executed in the ORDER you list them - the order MATTERS!
- If someone is looking information about Midnight, Blockchain → WEB_SEARCH_OR_RAG
- If you're actively participating in a conversation and the message continues that thread → WEB_SEARCH_OR_RAG
- Common patterns:
 - For requests requiring tool use: REPLY, CALL_MCP_TOOL (acknowledge first, then gather info)
 - For task execution: REPLY, SEND_MESSAGE or REPLY, EVM_SWAP_TOKENS (acknowledge first, then do the task)
 - For multi-step operations: REPLY, ACTION1, ACTION2 (acknowledge first, then complete all steps)
- REPLY is used to acknowledge and inform the user about what you're going to do
- Follow-up actions execute the actual tasks after acknowledgment
- Use IGNORE only when you should not respond at all
- If you use IGNORE, do not include any other actions. IGNORE should be used alone when you should not respond or take any actions.

IMPORTANT PROVIDER SELECTION RULES:

- Only include providers if they are needed to respond accurately.
- If the message mentions images, photos, pictures, attachments, or visual content, OR if you see "(Attachments:" in the conversation, you MUST include "ATTACHMENTS" in your providers list

- If the message asks about or references specific people, include "ENTITIES" in your providers list
- If the message asks about relationships or connections between people, include "RELATIONSHIPS" in your providers list
- If the message asks about facts or specific information, include "FACTS" in your providers list
- If the message asks about the environment or world context, include "WORLD" in your providers list
- If no additional context is needed, you may leave the providers list empty.

IMPORTANT CODE BLOCK FORMATTING RULES:

- If `{{agentName}}` includes code examples, snippets, or multi-line code in the response, ALWAYS wrap the code with ````` fenced code blocks (specify the language if known, e.g., ````python`).
- ONLY use fenced code blocks for actual code. Do NOT wrap non-code text, instructions, or single words in fenced code blocks.
- If including inline code (short single words or function names), use single backticks (```) as appropriate.
- This ensures the user sees clearly formatted and copyable code when relevant.

First, think about what you want to do next and plan your actions. Then, write the next message and include the actions you plan to take.

`</instructions>`

<keys>

"thought" should be a short description of what the agent is thinking about and planning.

"actions" should be a comma-separated list of the actions {{agentName}} plans to take based on the thought, IN THE ORDER THEY SHOULD BE EXECUTED (if none, use IGNORE, if simply responding with text, use REPLY)

"providers" should be a comma-separated list of the providers that {{agentName}} will use to have the right context for responding and acting (NEVER use "IGNORE" as a provider - use specific provider names like ATTACHMENTS, ENTITIES, FACTS, KNOWLEDGE, etc.)

"text" should be the text of the next message for {{agentName}} which they will send to the conversation.

</keys>

<output>

Do NOT include any thinking, reasoning, or <think> sections in your response.

Go directly to the XML response format without any preamble or explanation.

Respond using XML format like this:

<response>

<thought>Your thought here</thought>

```
<actions>ACTION1,ACTION2</actions>

<providers>PROVIDER1,PROVIDER2</providers>

<text>Your response text here</text>

</response>
```

Compose state

These templates are utilized by the Bootstrap plugin, which controls the message processing flow. When there is no specific definition of a template within the character, Bootstrap defaults to using standard templates.

On the example of the **messageHandlerTemplate** you can see special placeholders. These placeholders are specific instructions that are replaced during the compose state step. The Compose State is an Eliza process that gathers information from the current context to complete the template.

Some placeholders are straightforward, such as **{{agentName}}**, which is replaced by "Nyx," the defined name for the agent. However, other placeholders are more complex. The **{{providers}}** placeholder is the most intricate; it takes all registered providers, filters out those marked as "dynamic" and "public," and then combines all relevant information.

Core *providers* from the Bootstrap Plugin include:

- **characterProvider** Agent personality (name, bio, traits)
- **timeProvider** Current date/time
- **knowledgeProvider** Knowledge base/documentation
- **recentMessagesProvider** Chat history for context (can be from Discord, chat, or Telegram)
- **actionsProvider** Available actions
- **factsProvider** Stored facts/user preferences
- **settingsProvider** Configuration settings
- **entitiesProvider** Conversation participants

But also we are able to create any new provider (and register via plugin) to add custom information into the final prompt.

5.2. Prompt Template Structure

ElizaOS prompts are generally structured using the following XML template:

```
XML
<task></task>
<providers></providers>
<instructions></instructions>
<keys></keys>
<output></output>
```

Special parameters like `{{agentName}}` or `{{providers}}` are placeholders for dynamic content that is injected into the prompt before it is sent to the Large Language Model (LLM). This content is gathered by the registered providers, which include information such as conversation history, available actions, and knowledge. Prompts

Prompts are an important part of the system since it defines the task along with all information. Prompt A Structure

Task:

There are some Decide on behalf of `{{agentName}}` whether they should respond to the message, ignore it, or stop the conversation.

Providers:

All providers are registered for all plugins. They gather information and return text for injecting into the prompt. Some of this information comes from core behavior, such as conversation history, actions, knowledge, character definition, message examples, etc.

Instructions:

These outline how the LLM should respond and how it will make its decision.

Response:

The response is in the format expected by the Bootstrap plugin, usually as XML with tags like ***name***, ***reasoning***, and ***action***. With that action, Bootstrap can continue with the flow. Prompt B Structure

The main difference between Prompt A and Prompt B is the task and the expected result.

Task:

Generate dialogue and actions for the character {{agentName}}. Typically, respond with REPLY and the message to post in the chat.

Response:

The response is in the format expected by the Bootstrap plugin, usually as XML with tags like ***Thought***, ***Action***, ***Text***, etc. Prompt C

Regarding Prompt C sent to RAG, it contains the raw user question, while the historical messages are handled by a backend parameter that identifies the conversation, referred to as the ***RoomId*** parameter. This is an optimization that drastically reduces the number of tokens and increases the speed of the response to the user.

Prompt A Structure

Task:

Decide on behalf of {{agentName}} whether they should respond to the message, ignore it, or stop the conversation.

Providers:

All providers are registered for all plugins. They gather information and return text for injecting into the prompt. Some of this information comes from core behavior, such as conversation history, actions, knowledge, character definition, message examples, etc.

Instructions:

These outline how the LLM should respond and how it will make its decision.

Response:

The response is in the format expected by the Bootstrap plugin, usually as XML with tags like name, reasoning, and action. With that action, Bootstrap can continue with the flow.

Prompt B - Search Gate Template Prompt

Purpose: This prompt consolidates the necessary conversational and contextual data required by the search gate service to formulate an accurate and relevant response.

Content: It is structured to include:

Component	Placeholder	Description
User Question	<code>{{question}}</code>	The raw, unadulterated text of the current user query.
Conversation Context	<code>{{messages}}</code>	A collection of historical messages from the ongoing conversation, providing context for the current query.
Previous Reasoning	<code>{{reasoning}}</code>	The internal thought process or rationale generated by the LLM in a preceding step (e.g., during the execution of Prompt A or B), which guides the search strategy.

This structure ensures the search gate has both the immediate user intent and the larger context needed to effectively utilize its knowledge sources..

Search Gate Template Prompt

```
# Reasoning {{reasoning}}
# Conversation {{messages}}
# User question {{question}}
```

